ERREFAKTORI ZAZIOA

Software Ingeniaritza II

2026-2027

Alain del Ser

Aimar Goñi

Aurkibidea

1. Write simple units of code..... 3
2. Write simple units of code..... 7
3. Duplicate code..... 11
4. Keep unit interfaces small..... 14

1. Write simple units of code

Alain del Ser

Hasierako kodea:

```
public void declineReport(int reportID) {
	db.getTransaction().begin();
	Report r = db.find(Report.class, reportID);
	if (r == null) {
		db.getTransaction().commit();
		return;
	}
	Registered reg = r.getUser();
	Sale sale = r.getSale();
	if (reg != null) reg.getReports().remove(r);
	if (sale != null) sale.getReports().remove(r);
	db.remove(r);
	if (sale != null) {
		boolean reportsPending = false;
		for (Report rep : sale.getReports()) {
			if (!rep.isTreated()) {
				reportsPending = true;
				break;
			}
		}
		if (!reportsPending) {
			sale.setSaleStatus(SaleStatusType.ON_SALE);
		}
	}
	db.getTransaction().commit();
}
```

Hasierako kodean ikusi daitekeen moduan metodoaren luzeera 20 lerro ingurukoa da; *Building Maintainable Software* liburuaren arabera hau *issue* edo *bad smell* bat da kodearen kalitateari erreparatuz. Mantentze lana eta ulergarritasuna hobetzeko, bi metodo laguntzaile pribatu sortuko ditugu metodo honen bi ataza nagusiak desberdintzeko, jatorrizko metodoaren luzeera erdira jaitsiz. Hau egiteko behar den kodea aukeratu eta Refactor → Extract Method aukeratu dugu bi aldietan.

Errefaktorizatuko kodea:

```
public void declineReport(int reportID) {
	db.getTransaction().begin();
	Report r = db.find(Report.class, reportID);
	if (r == null) {
		db.getTransaction().commit();
		return;
	}
	Sale sale = removeReportFromUserSalesDatabase(r);
	if (sale != null) {
		restoreStatusIfNoPending(sale);
	}
	db.getTransaction().commit();
}

private Sale removeReportFromUserSalesDatabase(Report r) {
	Registered reg = r.getUser();
	Sale sale = r.getSale();
	if (reg != null) reg.getReports().remove(r);
	if (sale != null) sale.getReports().remove(r);
	db.remove(r);
	return sale;
}

private void restoreStatusIfNoPending(Sale sale) {
	boolean reportsPending = false;
	for (Report rep : sale.getReports()) {
		if (!rep.isTreated()) {
			reportsPending = true;
			break;
		}
	}
	if (!reportsPending) {
		sale.setSaleStatus(SaleStatusType.ON_SALE);
	}
}
```

Aimar Goñi

Hasierako kodea:

```
public void acceptOffer(Offer offer) throws NotEnoughMoneyException{
	db.getTransaction().begin();
	Offer o = db.find(Offer.class, offer.getOfferId());
	Request r = o.getRequest();
	o.setOfferStatus(OfferStatusType.ACCEPTED);
	r.setRequestStatus(RequestStatusType.COMPLETED);
	String offererEmail = o.getRegistered().getEmail();
	String requesterEmail = r.getRequester().getEmail();
	Registered requester = db.find(Registered.class, requesterEmail);
	Registered offerer = db.find(Registered.class, offererEmail);
	Double price = o.getPrice();
	if (price > requester.getBalance()) {
		throw new NotEnoughMoneyException();
	}
	Sale s = offerer.addSale(r.getTitle(), r.getDescription(),
		o.getStatus(),(float)o.getPrice(), UtilDate.trim(new Date()), null);
	s.setSaleStatus(SaleStatusType.BOUGHT);
	ArrayList<Sale> saleList = new ArrayList<>();
	saleList.add(s);
	double newBuyerBalance  = requester.getBalance() - price;
	double newSellerBalance = offerer.getBalance()   + price;
	requester.addToBought(saleList);
	requester.setBalance(newBuyerBalance);
	requester.addToMovements(MovementType.BUY,price,newBuyerBalance, saleList);
	offerer.setBalance(newSellerBalance);
	offerer.addToMovements(MovementType.SELL,price,newSellerBalance, saleList);
	db.getTransaction().commit();
}
```

Hasierako kodeak 30 kode lerro inguru ditu, eta hau *Building Maintainable Software* liburua jarraituz *bad smell* edo *issue* bat da, mantentze lanak eta ulergarritasuna oztopatuz. Hau ekiditeko eta hobetzeko, metodo honetatik bi metodo pribatu laguntzaile aterako ditugu, offer sortzeko metodo bat eta diru etabar mugimenduak kudeatzeko beste metodo bat. Honetarako Eclipsek eskaintzen duen errefaktorizazioa erabiliko dugu, *extract method.* Honerratako, nahi dugun kodea aukeratu, eskubiko botoia sakatu eta `Refactor` → `Extract Method` egingo dugu.

Errefaktorizatuko kodea:

```
public void acceptOffer(Offer offer) throws NotEnoughMoneyException{
	db.getTransaction().begin();
	Offer o = db.find(Offer.class, offer.getOfferId());
	Request r = o.getRequest();
	o.setOfferStatus(OfferStatusType.ACCEPTED);
	r.setRequestStatus(RequestStatusType.COMPLETED);
	String offererEmail = o.getRegistered().getEmail();
	String requesterEmail = r.getRequester().getEmail();
	Registered requester = db.find(Registered.class, requesterEmail);
	Registered offerer = db.find(Registered.class, offererEmail);
	Double price = o.getPrice();

	if (price > requester.getBalance()) {
		throw new NotEnoughMoneyException();
	}

	ArrayList<Sale> saleList = addOffer(o, r, offerer);
	manageMovements(requester, offerer, price, saleList);
	db.getTransaction().commit();
}

private void manageMovements(Registered requester, Registered offerer,
   Double price, ArrayList<Sale> saleList) {
	double newBuyerBalance  = requester.getBalance() - price;
	double newSellerBalance = offerer.getBalance()   + price;
	requester.addToBought(saleList);
	requester.setBalance(newBuyerBalance);
	requester.addToMovements(MovementType.BUY,price,newBuyerBalance,
	   saleList);
	offerer.setBalance(newSellerBalance);
	offerer.addToMovements(MovementType.SELL,price,newSellerBalance,
	   saleList);
}

private ArrayList<Sale> addOffer(Offer o, Request r, Registered offerer) {
	Sale s = offerer.addSale(r.getTitle(),
	   r.getDescription(),o.getStatus(),
	   (float) o.getPrice(), UtilDate.trim(new Date()), null);
	s.setSaleStatus(SaleStatusType.BOUGHT);
	ArrayList<Sale> saleList = new ArrayList<>();
	saleList.add(s);
	return saleList;
}
```

2. Write simple units of code

Alain del Ser

Hasierako kodea:

```
public boolean buySale(String mail, List<Integer> saleNumbers) throws
NotEnoughMoneyException{
	db.getTransaction().begin();

	Registered buyer = db.find(Registered.class, mail);
	ArrayList<Sale> sales = new ArrayList<Sale>();
	Sale first = db.find(Sale.class, saleNumbers.get(0));
2 ←	if(first == null || buyer == null) {
		db.getTransaction().rollback();
		return false;
	}
	first.setSaleStatus(SaleStatusType.BOUGHT);
	sales.add(first);
	float totalPrize = first.getPrice();
	Registered seller = first.getSeller();
	int i = 1;
3 ←	while(i < saleNumbers.size()) {
		Sale s = db.find(Sale.class, saleNumbers.get(i));
5 ←		if(s == null || seller != s.getSeller()) {
			db.getTransaction().rollback();
			return false;
		}
		s.setSaleStatus(SaleStatusType.BOUGHT);
		sales.add(s);
		totalPrize += s.getPrice();
		i++;
	}
6 ←	if (totalPrize > buyer.getBalance()) {
		db.getTransaction().rollback();
		throw new NotEnoughMoneyException();
	}
	double newBuyerBalance = buyer.getBalance()-totalPrize;
	buyer.addToBought(sales);
	buyer.setBalance(newBuyerBalance);

	buyer.addToMovements(MovementType.BUY,totalPrize,newBuyerBalance,sales);
	double newSellerBalance = seller.getBalance()+totalPrize;
	seller.setBalance(newSellerBalance);

	seller.addToMovements(MovementType.SELL,totalPrize,newSellerBalance,sales);
	cleanWishLists(sales);
	db.getTransaction().commit();
	return true;
}
```

Hasieran metodo honen konplexutasun ziklomatikoa estalpen anitza kontsideratuz 6+1-eko konplexutasun ziklomatikoa izango genuke. Lehen aipatutako liburuaren arabera, metodoek ez dute 4 adarkatze baino gehiago izan behar. Beraz, *bad smell* hau zuzentzeko kodea funtzio laguntzaileetara aterako dugu Eclipsel eskaintzen duen Extract Method-en laguntzatz. Lehenik eta behin, hasierako while-a produktu guztiak procesatzen dituena metodo pribatu batera pasa dugu, bertan kalkulatzen den saskiaren prezioa itzuliz. Produktu guztiak saltzaile berdinarenak ez badira -1 eko prezioa itzuliko du eta hau gehi prezio nahikoa ez izatea beste metodo batera atera dugu, balizko prezioa dela berifikatzen duena. Azkeneko metodo laguntzailea, mugimenduak sortzen dituena, ez du konplexutasunik gehitzen baina hau sortzea aproposa iruditu zaigu, aurreko bad smell-a ekiditzeko aukera garbia ikusi baitugu.

```
public boolean buySale(String mail, List<Integer> saleNumbers) throws
NotEnoughMoneyException{
	db.getTransaction().begin();
	Registered buyer = db.find(Registered.class, mail);
	ArrayList<Sale> sales = new ArrayList<Sale>();
	Sale first = db.find(Sale.class, saleNumbers.get(0));
2←	if(first == null || buyer == null) {
		db.getTransaction().rollback();
		return false;
	}
	first.setSaleStatus(SaleStatusType.BOUGHT);
	sales.add(first);
	float totalPrize = first.getPrice();
	Registered seller = first.getSeller();
	totalPrize = proccesSales(saleNumbers, sales, totalPrize, seller);
3←	if(!verifyTotalPrice(buyer, totalPrize)) return false;
	createBalances(buyer, sales, totalPrize, seller);
	cleanWishLists(sales);
	db.getTransaction().commit();
	return true;
}
```

```
private float proccesSales(List<Integer> saleNumbers, ArrayList<Sale> sales, float
totalPrize, Registered seller) {
	int i = 1;
	while(i < saleNumbers.size()) {
		Sale s = db.find(Sale.class, saleNumbers.get(i));
		if(s == null || seller != s.getSeller()) {
			db.getTransaction().rollback();
			return -1.0f;
		}
		s.setSaleStatus(SaleStatusType.BOUGHT);
		sales.add(s);
		totalPrize += s.getPrice();
		i++;
	}
	return totalPrize;
}
private boolean verifyTotalPrice(Registered buyer, float totalPrize) throws
NotEnoughMoneyException {
	if(totalPrize == -1.0f) {
		return false;
	}
	if (totalPrize > buyer.getBalance()) {
		db.getTransaction().rollback();
		throw new NotEnoughMoneyException();
	}
	return true;
}
private void createBalances(Registered buyer, ArrayList<Sale> sales, float
totalPrize, Registered seller) {
	double newBuyerBalance = buyer.getBalance()-totalPrize;
	buyer.addToBought(sales);
	buyer.setBalance(newBuyerBalance);
	buyer.addToMovements(MovementType.BUY,totalPrize,newBuyerBalance,sales);

	double newSellerBalance = seller.getBalance()+totalPrize;
	seller.setBalance(newSellerBalance);
	seller.addToMovements(MovementType.SELL,totalPrize,newSellerBalance,sales);
}
```

Aimar Goñi

Hasierako kodea:

```
public List<Offer> getOffers(String currentMail, String filter) {
      TypedQuery<Offer> query = db.createQuery("SELECT o FROM Offer o WHERE
        o.offerStatus = ?1", Offer.class);
      query.setParameter(1, OfferStatusType.WAITING);
      ArrayList<Offer> offers = new ArrayList<Offer>();
1 ←   for(Offer o:query.getResultList()) {
            Request r = o.getRequest();
2 ←         if(r.getRequester().getEmail().equals(currentMail) &&
3 ←            r.getRequestStatus().equals(RequestStatusType.AVAILABLE) &&
4 ←            r.getTitle().contains(filter)) {
                System.out.println("getOffers,currentmail:"+currentMail);
                offers.add(o);
            }
      }
      return offers;
}
```

Kode honek 5-eko konplexutasun ziklomatikoa du. *Building Maintainable Software* liburuak esaten duen bezala, 4-tik gorako konplexutasun ziklomatikoa izateak “arazoak” ekarri ditzake, kodea probatzea konplexuagoa bihurtuz, bide desberdin gehiago probatu behar baitira. Hau konpontzeko, berriz ere *Eclipsek* eskaintzen duen *extract method* errefaktorizazioa erabili dugu, if barruko baldintza guztiak aukeratuz eta hauek metodo pribatu laguntzaile batean ipiniz, *getOffers*-eko konplexutasuna 3-ra jaitsiz.

Errefaktorizatuko kodea:

```
public List<Offer> getOffers(String currentMail, String filter) {
      TypedQuery<Offer> query = db.createQuery("SELECT o FROM Offer o WHERE
        o.offerStatus = ?1", Offer.class);
      query.setParameter(1, OfferStatusType.WAITING);
      ArrayList<Offer> offers = new ArrayList<Offer>();
      for(Offer o:query.getResultList()) {
            Request r = o.getRequest();
            if(isMatchingRequest(currentMail, filter, r)) {
                  System.out.println("getOffers,currentmail:"+currentMail);
                  offers.add(o);
            }
      }
      return offers;
}

private boolean isMatchingRequest(String currentMail,String filter,Request r) {
      return r.getRequester().getEmail().equals(currentMail) &&
             r.getRequestStatus().equals(RequestStatusType.AVAILABLE) &&
             r.getTitle().contains(filter);
}
```

3. Duplicate code

Alain del Ser

Hasierako kodea:

```
public List<Sale> getPublishedSales(String desc, Date pubDate, String email) {
	System.out.println(">> DataAccess: getProducts=> from= " + desc);
	TypedQuery<Sale> query = db.createQuery("SELECT s FROM Sale s WHERE s.title
LIKE ?1 AND s.pubDate <= ?2 AND (s.saleStatus = ?3 OR s.saleStatus = ?4)",
Sale.class);
	query.setParameter(1, "%" + desc + "%");
	query.setParameter(2, pubDate);
	query.setParameter(3, SaleStatusType.ON_SALE);
	query.setParameter(4, SaleStatusType.USER_REPORTED);
	List<Sale> sales = query.getResultList();
	ArrayList<Sale> ema = new ArrayList<Sale>();
	for (Sale s : sales) {
		if (!s.getSeller().getEmail().equals(email)) {
			ema.add(s);
		}
	}
	return ema;
}

public List<Sale> getPublishedSales(String desc, Date pubDate, String buyerEmail,
String sellerMail, ArrayList<Sale> basket) {
	System.out.println(">> DataAccess: getProducts=> from= " + desc);
	TypedQuery<Sale> query = db.createQuery("SELECT s FROM Sale s WHERE s.title
LIKE ?1 AND s.pubDate <= ?2 AND (s.saleStatus = ?3 OR s.saleStatus = ?4)" ,
Sale.class);
	query.setParameter(1, "%" + desc + "%");
	query.setParameter(2, pubDate);
	query.setParameter(3, SaleStatusType.ON_SALE);
	query.setParameter(4, SaleStatusType.USER_REPORTED);
	List<Sale> sales = query.getResultList();
	ArrayList<Sale> ema = new ArrayList<Sale>();
	for (Sale s : sales) {
		if (!s.getSeller().getEmail().equals(buyerEmail) &&
s.getSeller().getEmail().equals(sellerMail)) {
			if(!basket.contains(s))
				ema.add(s);
		}
	}
	return ema;
}
```

Kodean ikusi daitekeen moduan, bi metodo daude `getPublishedSales()` deitzen direnak, izan ere, parametro desberdinekin. Hau guk hartu genuen erabaki bat izan zen, saskira produkturik sartu aurretik produktu guztiak erakutsi behar direlako, eta beste edozein kasuan lehengo produktuaren saltzailearen produktuak bakarrik erakutsiko dira. Hala ere bi metodoen hasiera berdina da, beraz kodea duplikatzea ekiditzeko hasiera metodo laguntzaile batera aterako dugu Eclipseren `Refactor → Extract Method` aukeraren bidez. Hau bakarrik behin egin dugu, beste metodoan automatikoki aplikatu da. Horrela kodea ez dugu errepikatuko, mantenua etorkizunean errazteko eta hau egitean gertatu litezkeen akatsak edukitzeko aukera murriztuz.

Errefakturizatutako kodea:

```
public List<Sale> getPublishedSales(String desc, Date pubDate, String email) {
	List<Sale> sales = querySales(desc, pubDate);
	ArrayList<Sale> ema = new ArrayList<Sale>();
	for (Sale s : sales) {
		if (!s.getSeller().getEmail().equals(email)) {
			ema.add(s);
		}
	}
	return ema;
}
private List<Sale> querySales(String desc, Date pubDate) {
	System.out.println(">> DataAccess: getProducts=> from= " + desc);
	TypedQuery<Sale> query = db.createQuery("SELECT s FROM Sale s WHERE s.title
LIKE ?1 AND s.pubDate <= ?2 AND (s.saleStatus = ?3 OR s.saleStatus = ?4)",
Sale.class);
	query.setParameter(1, "%" + desc + "%");
	query.setParameter(2, pubDate);
	query.setParameter(3, SaleStatusType.ON_SALE);
	query.setParameter(4, SaleStatusType.USER_REPORTED);
	List<Sale> sales = query.getResultList();
	return sales;
}

public List<Sale> getPublishedSales(String desc, Date pubDate, String buyerEmail,
String sellerMail, ArrayList<Sale> basket) {
	List<Sale> sales = querySales(desc, pubDate);
	ArrayList<Sale> ema = new ArrayList<Sale>();
	for (Sale s : sales) {
		if (!s.getSeller().getEmail().equals(buyerEmail) &&
s.getSeller().getEmail().equals(sellerMail)) {
			if(!basket.contains(s))
				ema.add(s);
		}
	}
	return ema;
}
```

Aimar Goñi

Hasierako kodea:

```
public boolean toggleWishList(String mail, int saleNumber) {
	boolean listanDago = isInWishList(mail, saleNumber);
	db.getTransaction().begin();
	Registered seller = db.find(Registered.class, mail);
	Sale sale = db.find(Sale.class, saleNumber);
	if (seller == null || sale == null) {
		db.getTransaction().commit();
		return false;
	}
	if(!listanDago) seller.addToWishList(sale);
	else 		seller.removeFromWishList(sale);
	db.getTransaction().commit();
	return true;
}

public boolean isInWishList(String mail, int saleNumber) {
	db.getTransaction().begin();
	Registered seller = db.find(Registered.class, mail);
	Sale sale = db.find(Sale.class, saleNumber);
	if (seller == null || sale == null) {
		db.getTransaction().commit();
		return false;
	}
	db.getTransaction().commit();
	return seller.getWishList().contains(sale);
}
```

Jatorrizko kodeak egitura eta datu-basearekiko kontsultak bikoizten zituen `toggleWishList()` eta `isInWishList()` metodoetan. Hori konpontzeko, `isInWishList()` metodoko transakzioa kendu da, metodoak datu basearen irakurketa egiten baitu soilik. `toggleWishList()` metodoan, aldiz, bestearekiko deia ezabatu da, memorian kargatuta dauden objektuak zuzenean erabiliz (`seller.getWishList().contains(sale)`). Honek kode bikoiztua ezabatzen du eta datu-baserako eskaerak erdira murrizten ditu.

Errefaktorizatuko kodea:

```
public boolean toggleWishList(String mail, int saleNumber) {
	db.getTransaction().begin();
	Registered seller = db.find(Registered.class, mail);
	Sale sale = db.find(Sale.class, saleNumber);
	if (seller == null || sale == null) {
		db.getTransaction().commit();
		return false;
	}
	if(!seller.getWishList().contains(sale)) seller.addToWishList(sale);
	else 		seller.removeFromWishList(sale);
	db.getTransaction().commit();
	return true;
}
public boolean isInWishList(String mail, int saleNumber) {
	Registered seller = db.find(Registered.class, mail);
	Sale sale = db.find(Sale.class, saleNumber);
	if (seller == null || sale == null) {
		return false;
	}
	return seller.getWishList().contains(sale);
}
```

4. Keep unit interfaces small

Alain del Ser

Hasierako kodea:

dataAcces/DataAccess.java

```
public Sale createSale(String title, String description, int status, float price,
Date pubDate, String sellerEmail, File file) throws  FileNotUploadedException,
MustBeLaterThanTodayException, SaleAlreadyExistException {
      try {
            if(pubDate.before(UtilDate.trim(new Date()))) {
                  throw new MustBeLaterThanTodayException("...");
            }
            if (file==null)
                  throw new FileNotUploadedException("...");
            db.getTransaction().begin();
            Registered seller = db.find(Registered.class, sellerEmail);
            if (seller.doesSaleExist(title)) {
                  db.getTransaction().commit();
                  throw new SaleAlreadyExistException("...");
            }
            Sale  sale  =  seller.addSale(title,  description,  status,  price,
pubDate, file);
            db.persist(seller);
            db.getTransaction().commit();
            System.out.println("sale stored "+sale+ " "+seller);
            return sale;
      } catch (NullPointerException e) {
            e.printStackTrace();
            db.getTransaction().commit();
            return null;
      }
}
```

businessLogic/BLFacadeImplementation.java

```
public void createSale(String title, String description,int status, float price,
Date pubDate, String sellerEmail, File file) throws  FileNotUploadedException,
MustBeLaterThanTodayException, SaleAlreadyExistException {
            dbManager.open();
            Sale product=dbManager.createSale(title, description, status, price,
pubDate, sellerEmail, file);
            dbManager.close();
}
```

gui/CreateSaleGUI.java

```
public class CreateSaleGUI extends JFrame {
. . . . . .
facade.createSale(fieldTitle.getText(),  fieldDescription.getText(),  numStatus,
price,  UtilDate.trim(jCalendar.getDate()), sellerMail, targetFile);
. . . . . .
}
```

Kodean ikusi daitekeen moduan `DataAccess`-eko `createSale()` metodoa 7 parametro ditu. Kodea ona garatzeko gomendatzen da funtzioek 4 parametro baino gehiago ez izatea, eta hau lortzeko erlazionatuta dauden parametroak klase batean taldekatzea gomendatzen da. Hau egiteko Eclipsek duen `Refactor` → `Introduce Parameter Object` aukera erabili dugu, parametro guztiak sellerEmail izan ezik taldekatzeko. Aukera honek `domain` paketean `CreateSaleParameter` izeneko klase bat sortu du eraikitzaile hutsarekin eta setter banarekin atributu bakoitzerako. Eraikitzailean 6 parametro jarriz gero *bad smell* hau beste toki batera mugitzen arituko ginateke.

Kodearen signatura aldatu dugunez, deiak ere errefektorizatu behar dira, kasu honetan `BLFacadeImplementation`-etik egiten da dei bakarra eta hau aldatzeko `Refactor` → `Change Method Signature` egin dugu; interfazean ere signatura automatikoki aldatzeko, parametro guztiak klase honengatik ordeztuz. Azkenik, `CreateSaleGUI` klasean lehen zegoen deia aldatu behar da. Parametroak zuzenenan jarri ordez, `CreateSaleParameter` sortzen da eta setter-en bidez balioak esleitzen zaizkio. Gero objektu hau email-arekin batera, facadeko `createSale()` funtzioari pasatzen zaio, lehen bezala.

Errefaktorizatutako kodea:

dataAcces/DataAccess.java

```
public Sale createSale( String sellerEmail, CreateSaleParameter parameterObject)
throws              FileNotUploadedException,       MustBeLaterThanTodayException,
SaleAlreadyExistException {
      try {
            if(parameterObject.pubDate.before(UtilDate.trim(new Date())))
                  throw new MustBeLaterThanTodayException("..."));
            if (parameterObject.file==null)
                  throw new FileNotUploadedException("..."));
            db.getTransaction().begin();
            Registered seller = db.find(Registered.class, sellerEmail);
            if (seller.doesSaleExist(parameterObject.title)) {
                  db.getTransaction().commit();
                  throw new SaleAlreadyExistException("..."));
            }
            Sale  sale  =  seller.addSale(parameterObject.title,  parameterObject.
description,            parameterObject.status,            parameterObject.price,
parameterObject.pubDate, parameterObject.file);
            db.persist(seller);
            db.getTransaction().commit();
            return sale;
      } catch (NullPointerException e) {
            e.printStackTrace();
            db.getTransaction().commit();
            return null;
      }
}
```

businessLogic/BLFacadeImplementation.java

```
public void createSale(String sellerEmail, CreateSaleParameter data) throws
FileNotUploadedException, MustBeLaterThanTodayException, SaleAlreadyExistException
{
	dbManager.open();
	Sale product =dbManager.createSale(sellerEmail, data);
	dbManager.close();
}
```

gui/CreateSaleGUI.java

```
public class CreateSaleGUI extends JFrame {
. . . . . .
	CreateSaleParameter data = new CreateSaleParameter();
	data.setDescription(fieldDescription.getText());
	data.setFile(targetFile);
	data.setPrice(price);
	data.setPubDate(UtilDate.trim(jCalendar.getDate()));
	data.setStatus(numStatus);
	data.setTitle(fieldTitle.getText());
	facade.createSale(sellerMail, data);
. . . . . .
}
```

domain/CreateSaleParameter.java

```
public class CreateSaleParameter {
	public String title;
	public String description;
	public int status;
	public float price;
	public Date pubDate;
	public File file;
	public CreateSaleParameter() {
		super();
	}
	public void setTitle(String title) {
		this.title = title;
	}
	public void setDescription(String description) {
		this.description = description;
	}
	//Beste setter-ak berdin
}
```

Aimar Goñi

Hasierako kodea:

dataAccess/DataAcces.java

```
public void makeOffer(String offererMail, Request request, double price, int
status, String description) {
	db.getTransaction().begin();
	Registered reg = db.find(Registered.class,offererMail);
	if (reg== null) return;
	reg.addOffer(request,status,price,description);
	db.getTransaction().commit();
}
```

domain/Registered.java

```
public void addOffer(Request request,int status, double price, String description)
{
	Offer o = new Offer(price,description,status,this,request);
	offers.add(o);
	request.addOffer(o);
}
```

businessLogic/BLFacadeImplementation.java

```
public void makeOffer(String offererMail, Request request, double price, int
status, String description) {
	dbManager.open();
	dbManager.makeOffer(offererMail, request, price,status, description);
	dbManager.close();
}
```

Jatorrizko `makeOffer()` metodoak 5 parametro zituen, gehienezko 4ko muga gaindituz. Negozio Logikako geruzak (*BL*) ezin duenez datu-baseko *Offer* osoa zuzenean sortu, erabiltzailearen eta eskaeraren Instantziak `DataAccess`-en lortzen direlako, `Introduce Parameter Object` errefaktorizazioa erabili dugu. Datu askeak pasatu beharrean, eskatutako ezaugarriak (`price, status, description`) `OfferData` izeneko *transferentzia-objektu (DTO)* sinple batean bateratu dira. Horrela, `makeOffer` metodoaren parametroak 5etik 3ra murriztu dira, eta `addOffer()` metodoarenak 4tik 2ra.

DataAccess-eko aldaketak direla eta *BLFacdeImplementation* klasean aldeketa txiki bat egitera behartu gaitu, `OfferData` motako isntantzia bat sortuz eta *DataAccess*-eko `makeOffer()` metodoari pasaz, honek BL logikako geruzak hausten dituela dirudi, baina domain paketean sortutako `OfferData` klaseak ez du ezer berririk egiten, atributu batzuk garraiatu soilik, hau da esan bezala DTO objektu bat denez, salbuespen moduan tratatzen da. Aldaketa hauekin interfazeak txikiagoak eta irakurgarriagoak izatea lortu dugu.

Errefaktorizatuko kodea:

dataAccess/DataAcces.java

```
public void makeOffer(String offererMail, Request request, OfferData data) {
	db.getTransaction().begin();
	Registered reg = db.find(Registered.class,offererMail);
	if (reg== null) return;
	reg.addOffer(request,data);
	db.getTransaction().commit();
}
```

domain/Registered.java

```
public void addOffer(Request request,OfferData data) {
    Offero=new Offer(data.price,data.description,data.status,this,request);
    offers.add(o);
    request.addOffer(o);
}
```

businessLogic/BLFacadeImplementation.java

```
public void makeOffer(String offererMail, Request request, double price, int
status, String description) {
	OfferData data = new OfferData(price, status, description);
	dbManager.open();
	dbManager.makeOffer(offererMail, request, data);
	dbManager.close();
}
```

domain/OfferData.java

```
package domain;
public class OfferData {
	public double price;
	public int status;
	public String description;

	public OfferData(double price, int status, String description) {
		this.status = status;
		this.price = price;
		this.description = description;
	}

}
```